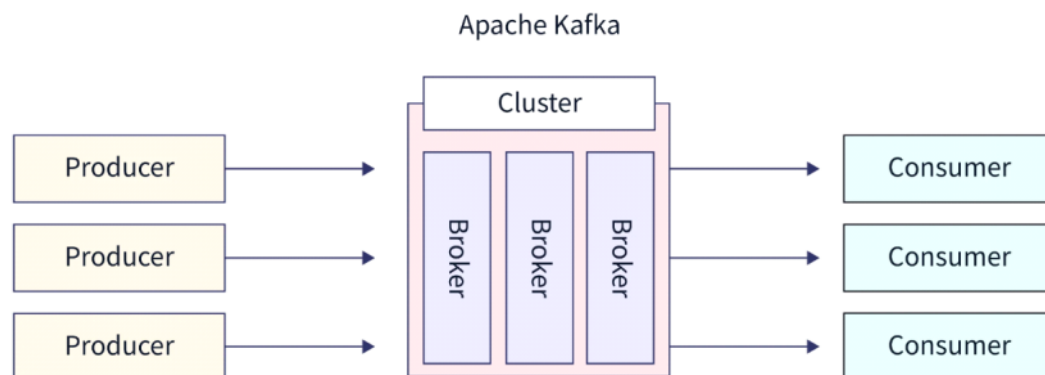


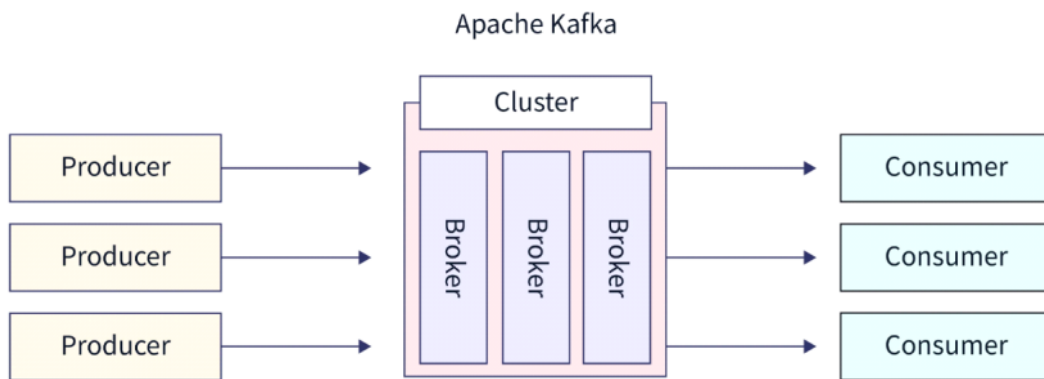
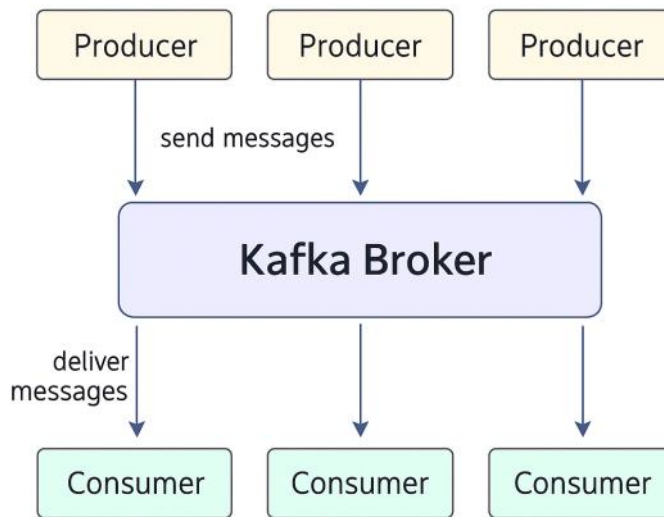
What is a Kafka Cluster?

- A Kafka Cluster is a **group of Kafka Brokers working together**.
- They **receive messages from producers**, store them, and **make them available to consumers**.
- Instead of running Kafka on one machine, a cluster uses **multiple brokers**.
- This helps the system **handle more data** and **serve more clients**.
- The cluster keeps running **even if one broker fails**.



What is a Kafka Broker?

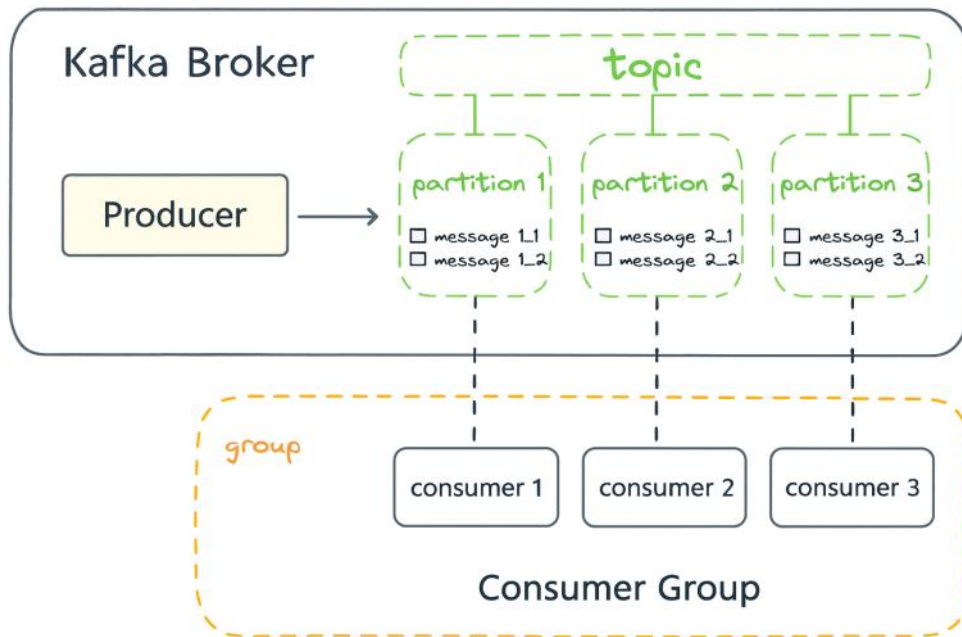
- A Kafka Broker is a **server inside Apache Kafka** that receives messages from producers, stores those messages in partitions, and delivers them to consumers when they request them.
- It is the **main component** that keeps Kafka running.
- The broker **manages data**, handles **read and write requests**, and works with other brokers to form a **Kafka Cluster**.



What Is a Kafka Topic and What Are Partitions?

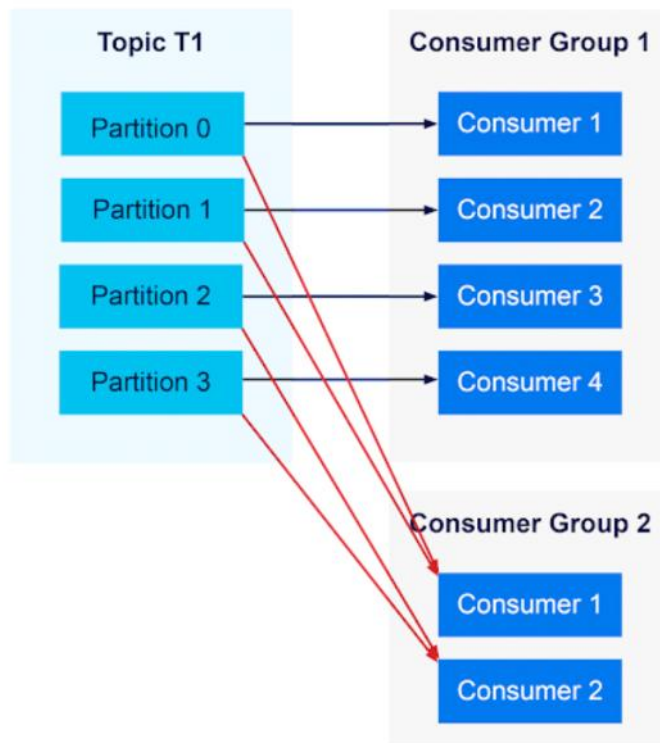
- A Kafka Topic is a **named channel** where Kafka stores messages.
- A producer never sends data directly to a broker — it always sends data **into a topic**.
- A topic is like a **category or folder** for related messages (orders, payments, notifications).
- A Kafka Topic is divided into **smaller units called partitions**.
- A partition is an **ordered message log** where messages are stored **in the exact order they arrive**.
- Kafka uses **multiple partitions** to achieve **high performance** and **parallel message**

processing.



What Are Consumer Groups?

- A Consumer Group is a **set of consumers** that work together to read messages from a Kafka topic.
- When consumers belong to the **same group**, Kafka **divides the topic's partitions** among them.
- Each consumer gets **only its share** of the partitions.
- This allows Kafka to process messages **faster, in parallel**, and **without duplicating data** inside the same group.



How Does Kafka Achieve High Throughput and Low Latency?

- Kafka writes data **sequentially to disk**, which is extremely fast.
- Kafka uses **zero-copy**, sending data from disk to network without extra copying.
- Kafka stores data in **partitions**, allowing many reads and writes at the same time.
- Kafka uses the **page cache**, so most reads come directly from memory.
- Kafka supports **batching**, where producers and consumers send or read many messages in one request.
- Kafka **scales horizontally** by adding more brokers and partitions.
- Kafka uses a **lightweight protocol**, which keeps latency low and throughput high.

What Is Kafka Replication?

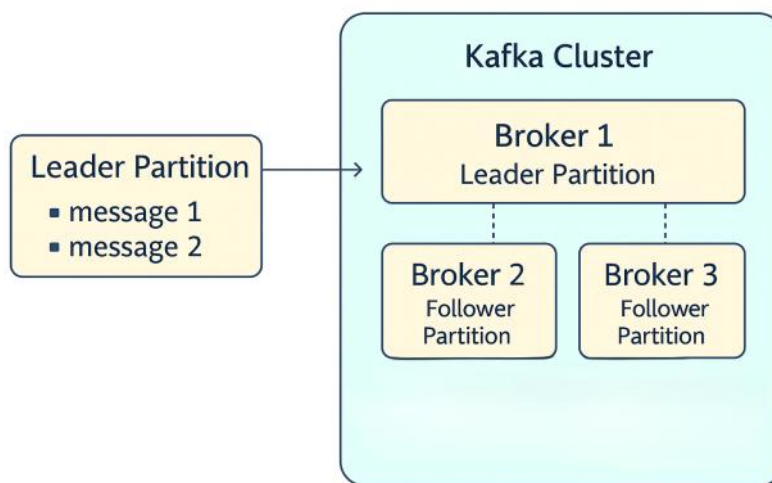
Kafka Replication keeps copies of data across multiple brokers to prevent data loss.

It ensures your data remains safe even if a broker fails.

Each partition has **one leader** and **multiple followers**.

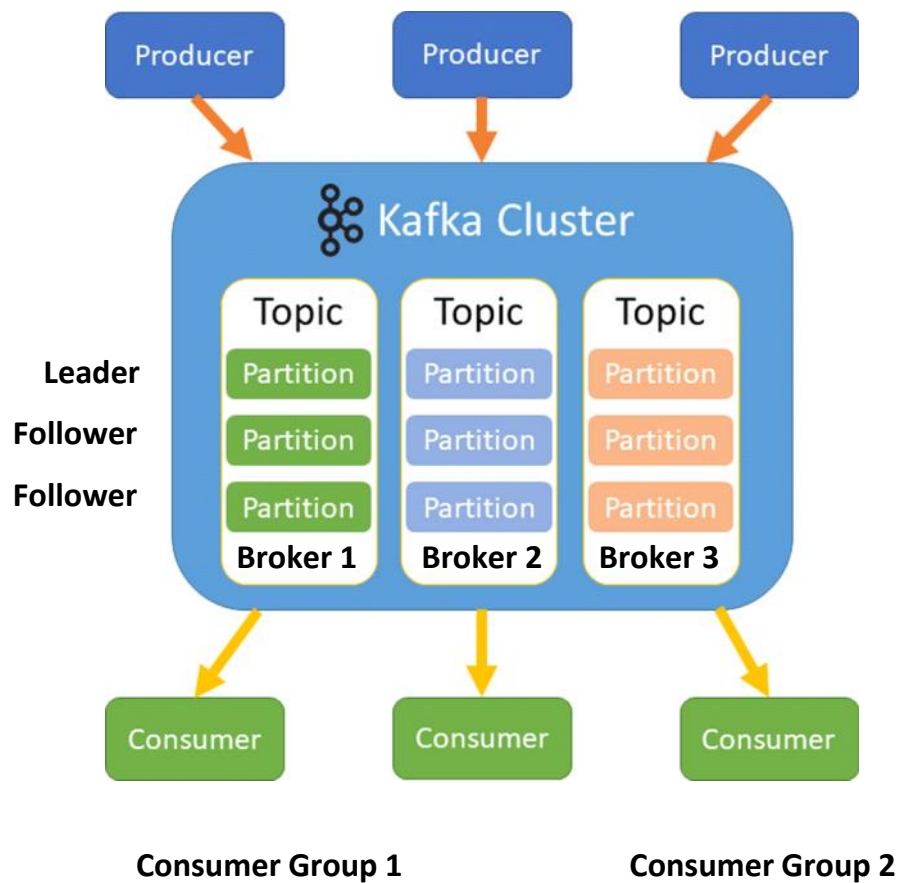
The **leader** handles all reads and writes.

The **followers** replicate data from the leader in real time.



Kafka Architecture

Kafka's architecture is designed to handle massive amounts of data with very high speed, very low latency, and strong fault tolerance.



What is a Kafka Producer?

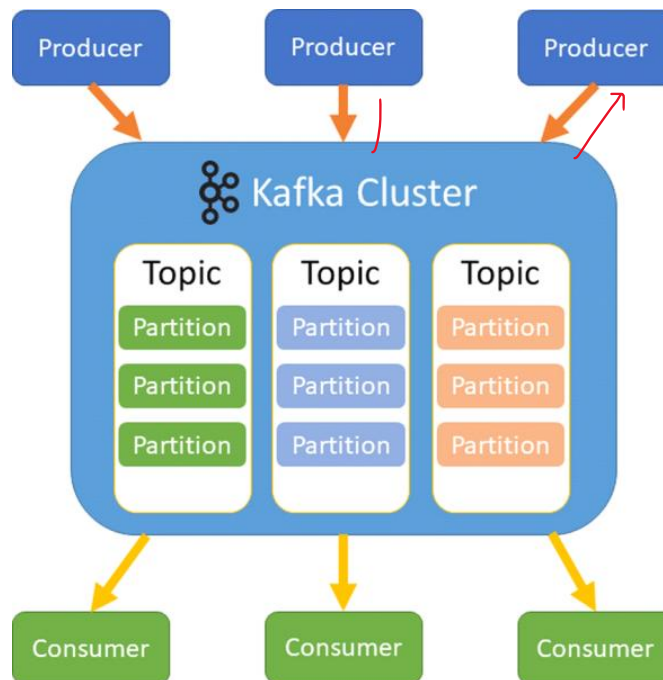
A Kafka Producer is an application that sends messages into Kafka.

Any service that generates new data and publishes it for others to use acts as a producer.

It could be an order service, a payment system, a logging service, or even a mobile app sending user activity.

The producer's job is simple: it creates the message and sends it to Kafka.

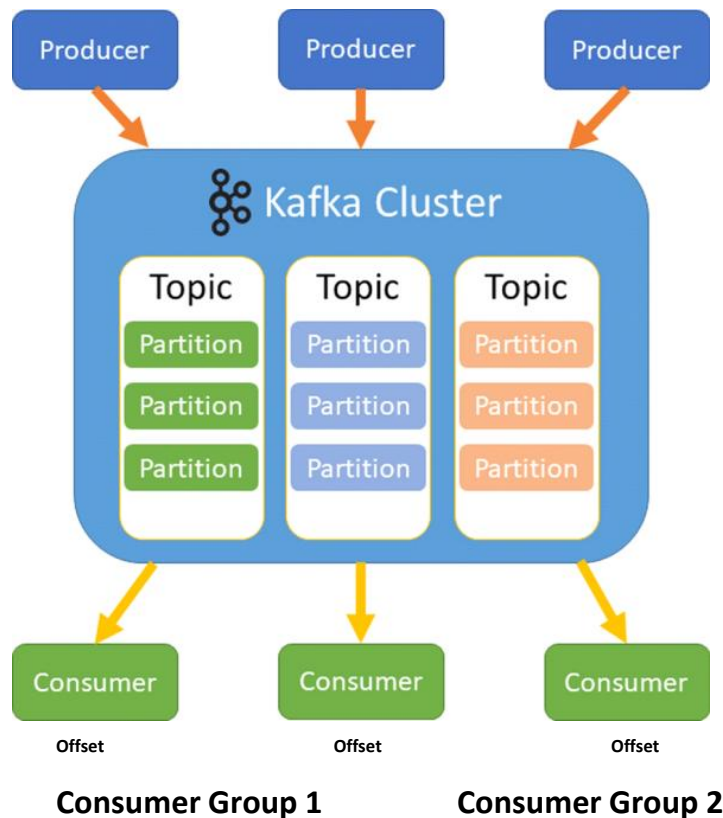
Partition
Key
Round-robin



Acknowledgment = 0
Acknowledgment = 1
Acknowledgment = all

What Is a Kafka Consumer?

- A Kafka Consumer is an application that reads messages from Kafka.
- Any service that needs to process data produced by another service acts as a consumer.
- Examples: order processors, analytics systems, notification services.
- The consumer's job: connect to Kafka, read messages from a topic, and process them in the correct order.



How Kafka Ensures Message Durability

- Message durability means **Kafka never loses your data**, even if brokers crash or servers restart.
- Kafka achieves durability through **several internal mechanisms** that protect messages from failures.
- These mechanisms ensure your data remains safe **no matter what happens** inside the cluster.

1. Messages Are Written to Disk Immediately

- Kafka **never stores messages in memory**.
- It writes every message **directly to disk** in an **append-only log file**.
- Sequential disk writes are **very fast**, and stored data **survives crashes and restarts**.

2. Replication Across Multiple Brokers

- Kafka keeps **multiple copies** of every partition on **different brokers**.

- Example: **Replication factor = 3** → 1 leader + 2 followers.
- Even if **one or two brokers fail**, your data remains safe on other replicas.
- Replication is the **core durability guarantee**.

3. In-Sync Replicas (ISR)

- Kafka tracks **In-Sync Replicas (ISR)** — followers fully caught up with the leader.
- Kafka **only acknowledges writes** when data is stored on leader **and** ISR followers.
- Ensures **no message loss**, even if the leader fails immediately after writing.

4. Acknowledgment Levels (acks Setting)

- Producer controls durability using **acks**.
- **acks = 0** → send and forget: fastest but unsafe.
- **acks = 1** → leader confirms write.
- **acks = all** → leader + all ISR replicas must store the message.
- **acks = all = strongest durability** (used in production).

5. Immutable Log Segments

- Kafka log files are **immutable**.
- Once written, messages are **never modified**, never overwritten, never rearranged.
- Immutability prevents corruption and keeps data safe.

6. Configurable Retention Policies

- Kafka can store data for **hours, days, months, or forever**.
- Even after consumers read messages, Kafka keeps them based on **retention settings**.
- Retention adds **another safety layer** for durability.

7. KRaft for Metadata Durability

- New Kafka versions use **KRaft mode** instead of ZooKeeper.

- Metadata is replicated across **multiple controller nodes**.
- This improves **metadata durability** and avoids metadata loss.

Final Summary

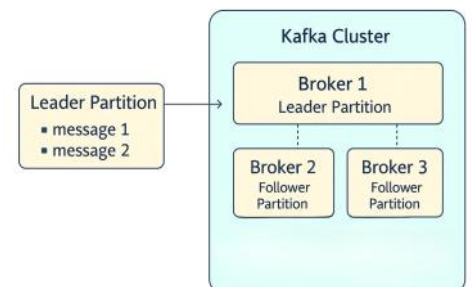
- Kafka ensures durability through:
 - **Writing messages to disk**
 - **Replication across brokers**
 - **ISR mechanism**
 - **acks levels**
 - **Immutable logs**
 - **Retention policies**
 - **KRaft metadata replication**
- These features make Kafka a **highly durable, fault-tolerant messaging system**.

How does Kafka handle fault tolerance?

- Fault tolerance means the system keeps working even when parts fail.
- Kafka is designed to survive broker failures, crashes, network issues, and server loss.
- Kafka continues running without losing data and without stopping message flow.

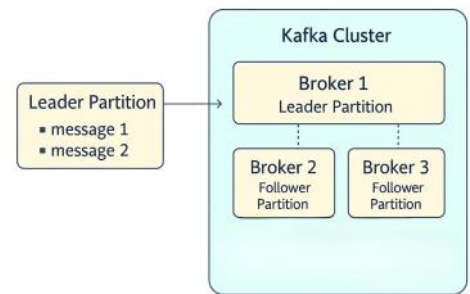
1. Partition Replication Across Brokers

- Fault tolerance starts with replication.
- Every partition is stored on multiple brokers.
- Example: replication factor = 3 → 1 leader + 2 followers
- If one broker fails, healthy copies still exist.
- Replication prevents message loss during failures.



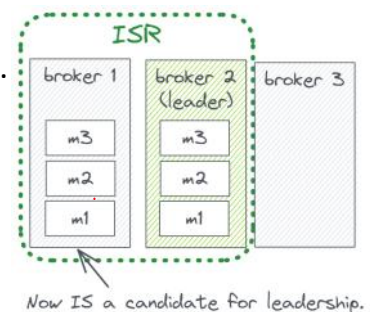
2. Leader and Follower Architecture

- Each partition has a leader + follower replicas.
- Leader handles all reads and writes.
- Followers continuously copy data from the leader.
- If the leader broker fails → a follower is promoted automatically.
- This automatic switch prevents downtime.



3. In-Sync Replicas (ISR)

- Kafka maintains replicas that are fully caught up with the leader.
- These replicas form the ISR list.
- Only ISR members can become leader during failure.
- Ensures new leader always has complete and latest data.
- Prevents promoting replicas that are behind or missing messages.



4. Acknowledgment Settings Ensure Safe Writes

- Producers control safety using **acks**.
- **acks = 1** → write only to leader (may lose messages if leader fails).
- **acks = all** → write stored in leader + all ISR followers.
- Strongest durability and fault-tolerance guarantee.
- Ensures at least one safe replica exists even after failure.

5. Automatic Rebalancing

- When brokers join or leave, Kafka redistributes partitions.
- If a broker fails → partitions assigned to healthy brokers.

- When broker returns or new broker added → data rebalanced.
- Keeps cluster balanced and reliable at all times.

6. KRaft Metadata Replication (Modern Kafka)

- Modern Kafka uses **KRaft mode** instead of ZooKeeper.
- Metadata is replicated across multiple controller nodes.
- Cluster keeps running even if one controller fails.
- Makes Kafka fault tolerance even stronger.

Summary — How Kafka Achieves Fault Tolerance

- Replication across brokers.
- Leader–follower promotion on failure.
- In-Sync Replicas ensure safe leaders.
- Acks guarantee safe writes.
- Automatic rebalancing on broker events.
- Metadata replication using KRaft.
- Together these mechanisms prevent downtime and data loss.